

Contracts: why the house is not on fire (i.e. why the status quo is tolerable)

Abstract

This paper provides an explanation why there's no particular panic to make design changes to Contracts, and how fears of unlimited UB are.. ..mistaken.

Assumption of unchecked contracts

Violations of unchecked contracts are Undefined Behavior. This causes severe heartburn to many audiences. However,

1. according to multiple implementation vendors, they are not likely (in fact, far from it) to assume contracts when `continuation==on`,
2. it's perfectly plausible that implementations will provide switches to decide when or if contracts are assumed,
3. ..and they do not need to assume anything in their default build mode, in any of their existing optimized mode, or anything that would necessarily break existing programs even if contracts are introduced.

In other words: if you want your program to be unaffected by contracts, build with contract checking disabled and turn continuation on. Or just disable contract assumptions via implementation-specific means (which may translate to doing nothing special).

Why did you bother co-authoring a proposal to change it, then?

Because I think it would be a better design to opt in to assumptions, rather than having them be a coupled part of the most-often used syntax. I also think we could ship C++20 without contract-based optimizations, and figure out how to add those later, perhaps in C++23.

I would also really prefer having a portable way to write contracts that are not ever assumed, and be able to have the source code I write mean that. With the status quo, I need to invent novel schemes for that, and some of said schemes might end up involving macros.

Future evolution paths

We haven't closed any evolution paths. They might need less convenient syntax than what is in the status quo, but that's perhaps more an inconvenience than a show-stopper problem.